

Quantifying and Improving Human-Likeness in AI-Generated CFOP Solutions for Rubik’s Cube: A Trigger-Biased Search Approach

[TODO: Ten tac gia]
[TODO: Don vi / Truong]
[TODO: email]

Abstract

Existing computational approaches to solving the Rubik’s Cube – including pattern-database search [4], group-theoretic optimal solvers [7, 3], and reinforcement learning [6] – optimize almost exclusively for move count, producing solutions that are correct but largely uninterpretable to human learners. In contrast, the human-designed CFOP method (Cross, F2L, OLL, PLL) trades move-count optimality for learnability, structured around a finite set of memorable move-sequence patterns (“algorithms”). We formalize the notion of *human-likeness* of a cube-solving plan in the vocabulary of human-aware AI planning [5, 1], propose a composite *Human-Likeness Index* (HLI) built from three measurable components (stage segmentability, named-algorithm pattern-conformity, and finger-trick trigger-overlap), and introduce *Trigger-Biased A* Search*, a heuristic-search modification that explicitly rewards search branches overlapping with a corpus of human finger-tricks. Pilot experiments (N=100 for the baseline comparison, N=60 for the Pareto sweep) show that biasing the F2L search heuristic increases trigger-overlap from a baseline of 9.0% to 25.5% at $\lambda = 1.0$ (Wilcoxon $p = 8.3 \times 10^{-8}$), at a move-count cost of only 7.4% (Wilcoxon $p = 0.0014$), revealing a favorable, though non-linear (in compute time), Pareto trade-off. We further outline a BotPrize-style [2] human-judge validation protocol to establish construct validity of HLI. This work is, to our knowledge, the first to apply explicability-style metrics to a large combinatorial search domain, bridging classical heuristic search AI with human-aware and believable-agent AI research.

1 Introduction

Solving the Rubik’s Cube has long served as a benchmark domain for heuristic search and combinatorial optimization [4, 7], and more recently for self-play reinforcement learning [6]. These approaches share a common objective: minimizing the number of moves (or maximizing win rate against an adversarial

scrambler), with no regard for whether the resulting solution corresponds to any recognizable pattern a human could learn, memorize, or explain.

Human speedcubers, in contrast, use structured methods such as CFOP (Cross, F2L, OLL, PLL), deliberately trading move-count optimality for *learnability*: each stage reduces to recognizing one of a bounded number of canonical “cases,” each paired with a fixed, named algorithm. The full CFOP method comprises 57 named OLL cases and 21 named PLL cases; a widely-used simplified variant (“2-look OLL/PLL”) solves each stage in two sequential passes using a much smaller subset of algorithms, at the cost of extra moves. Our system implements a verified table of all 21 PLL cases (Section 4.1) but only a 2-look OLL (2 generator algorithms, Sune/Anti-Sune, rather than the full 57); this asymmetry is itself informative for Pattern-conformity measurement (Section 4).

[TODO: Doan nay tom tat khoang trong nghien cuu – xemphan Related Work ben duoi, viet lai ngan gon o day, ket bang cau dan vao contributions]

Contributions. This paper makes the following contributions:

1. A formalization of *human-likeness* for cube-solving plans grounded in the human-aware planning notion of explicability [5, 1], instantiated as a composite **Human-Likeness Index (HLI)** with three measurable, reproducible components (Section 4).
2. **Trigger-Biased A* Search**: a minimal, backward-compatible modification to pattern-database-guided A* search for F2L that explicitly optimizes for human finger-trick overlap via a single interpretable hyperparameter λ (Section 4).
3. An empirical Pareto analysis of the human-likeness / move-count / compute-time trade-off induced by λ (Section 6).
4. A proposed, BotPrize-style [2] human-judge validation protocol for construct-validating HLI (Section 9), positioning this work for extension into a full empirical study.

2 Related Work

2.1 Computer-optimal Rubik’s Cube solvers

Korf’s IDA* with additive pattern databases [4] was the first method to find provably optimal solutions for arbitrary scrambles. Rokicki et al. [7] established that every cube state is solvable within 20 face turns (“God’s Number”), via large-scale coset-based search. Kociemba’s two-phase algorithm [3] trades strict optimality for tractable runtime, typically returning solutions within 1–4 moves of optimal, and is the baseline we use in Section 5. McAleer et al. [6] showed

a reinforcement-learning agent (DeepCubeA) can learn to solve the cube without built-in human knowledge, achieving near-optimal solution lengths. None of these methods produce plans that are segmentable, named, or otherwise interpretable in terms any human learner would recognize — they are, in our terminology, maximally *computer-like*.

2.2 Human-aware planning and explicability

A separate line of work in AI planning studies how autonomous agents should behave when observed or assisted by humans with an internal, possibly imperfect mental model of the agent. Kulkarni et al. [5] formalize *explicability* as minimizing the distance between an agent’s actual plan and the plan the human observer expects, and propose search modifications that bias planning toward more explicable trajectories — structurally analogous to the trigger-biased heuristic we propose in Section 4, but applied here to symbolic task planning rather than a large combinatorial puzzle domain. Chakraborti et al. [1] survey this landscape (explicability, legibility, predictability, transparency), which we adopt as the theoretical vocabulary for this paper. **[TODO: Neu tim them duoc bai lien quan (vd Zhang et al. 2017 ve legibility trong robot task planning), them 1-2 cau o day.]**

2.3 Human-likeness / believability of game-playing agents

Independently, the game-AI community has long studied how to make autonomous agents *appear* human. The 2K BotPrize competition [2] operationalized this as a Turing-test variant in which human judges rate both human and bot players on a 1–5 human-likeness scale, without being told which is which. **[TODO: 1 cau ve ket qua BotPrize 2012 (UT^2, MirrorBot vuot qua nguong) – da co du lieu tu timkiem web o buoc truoc, dien vao day, PARAPHRASE khong trich nguyen van].** A recent survey [8] frames human-like game AI research around two questions — how to model/implement human-like behavior, and how to measure its degree — which we adopt as the organizing structure for this paper’s two main contributions (Sections 4).

2.4 CFOP-specific tutoring systems

Closest to our work, the ALLURE system [9] pairs an automatic solver (DeepCubeA-based search over subgoals) with an interactive chatbot tutor to help learners understand explainable solutions to the White Cross sub-goal of CFOP; a recent pilot study [10] (unpublished manuscript at the time of writing) analyzes student engagement patterns within ALLURE via behavioral clustering, reporting that the beginner white-cross strategy requires substantially more moves than CFOP, which in turn requires more moves than fully optimized methods — directly illustrating the learnability/optimalty trade-off central to our formalization (Section 3). Our work differs in (a) scope — covering all four CFOP stages rather than Cross alone — and (b) mechanism — an explicit, tunable

search bias (Section 4) rather than post-hoc natural-language explanation of moves already found by an unconstrained solver.

3 Problem Formalization

Let G denote the Rubik’s Cube group ($|G| \approx 4.3 \times 10^{19}$), and let a *plan* $\pi = (m_1, \dots, m_k)$ be a sequence of face-turn generators solving a given scramble $s \in G$. Let M_H denote a *human expectation model*: in this paper, M_H is instantiated concretely as (i) the CFOP stage decomposition (Cross $\rightarrow$ F2L $\rightarrow$ OLL $\rightarrow$ PLL), (ii) a finite table of named, canonical algorithms for OLL/PLL (Section 4.1), and (iii) a corpus T of short move n-grams (“finger-tricks”) commonly executed as single motor units by human solvers (Section 4.1).

Following Kulkarni et al. [5], we define human-likeness as an (inverse) distance between π and the behavior M_H would consider natural. We instantiate this distance as a weighted sum of three normalized, independently-measurable components (rather than a single learned or hand-tuned distance function), described in Section 4.

4 Method

4.1 Human-Likeness Index (HLI)

Segmentability $S(\pi) \in \{0, 1\}$. Equals 1 iff π decomposes into four contiguous, independently verifiable sub-sequences satisfying the Cross, F2L, OLL, and PLL postconditions in order, and 0 otherwise (e.g. for plans that solve the cube via an undifferentiated optimal sequence, as computer-like solvers do by construction).

Pattern-conformity $P(\pi) \in [0, 1]$. Restricted to the OLL and PLL stages (where a finite, named case-to-algorithm mapping exists in CFOP), $P(\pi)$ is the fraction of solved stages whose corresponding sub-sequence exactly matches an entry in a verified table of canonical algorithms (Aa, Ab, $\dots$, Z for PLL; Sune/Anti-Sune generators for 2-look OLL) rather than an unstructured search fallback. **[TODO: Trich so lieu: X/Y PLL case verified, bang chung minh bang verify_and_build_table()]**.

Trigger-overlap $O(\pi) \in [0, 1]$. For the Cross and F2L stages (which lack a canonical case table in this system, being generated by unconstrained heuristic search), we measure the fraction of moves covered by a greedy longest-match segmentation of π against a seed corpus T of $|T| = 11$ common human finger-tricks (e.g. the “sexy move” $RU R' U'$, “sledgehammer” $R' F R F'$; full corpus in Appendix **[TODO: X]**). **[TODO: GHI RO trong bai: day la seed corpus thu cong, de xuat mo rong bang thong ke n-gram tren log solve nguoi that (vd CSTimer) o ban mo rong.]**

Composite HLI.

$$\text{HLI}(\pi) = w_S \cdot S(\pi) + w_P \cdot P(\pi) + w_O \cdot O(\pi), \quad w_S + w_P + w_O = 1$$

We use $(w_S, w_P, w_O) = (0.3, 0.4, 0.3)$ in this draft. We conducted a post-hoc sensitivity analysis using $N = 78$ paired scrambles (Group A vs. Group B, Experiment 1), measuring all three HLI components for *both* groups — including, critically, Group B’s trigger-overlap (measured directly on Kociemba’s actual move sequences, not assumed to be zero: mean 0.009 ± 0.046 , confirming empirically that an optimizer with no incentive toward human finger-tricks essentially never produces them by chance). Across all tested weight settings (five named settings plus a $w_S \in \{0, 0.2, \dots, 1\}$ sweep), the HLI gap between Group A and Group B remains large and strictly positive, ranging from 0.397 (trigger-heavy weighting) to 1.000 (segmentability-only weighting), with our chosen weights giving a gap of 0.653. This indicates the paper’s central claim — that AICFOP is substantially more human-like than a computer-optimal baseline — is robust to the specific choice of (w_S, w_P, w_O) within any reasonable range, though the *magnitude* of the gap is naturally weight-dependent.

4.2 Trigger-Biased A* Search

Our F2L solver uses A* with heuristic $h(n) = h_{\text{PDB}}(n) + \text{penalty}(n)$, where h_{PDB} is an admissible single-piece pattern-database estimate and $\text{penalty}(n)$ discourages disturbing already-solved pieces (Cross and prior F2L pairs). We introduce a third term:

$$f'(n) = g(n) + h_{\text{PDB}}(n) + \text{penalty}(n) - \lambda \cdot \text{bonus}(\text{path}(n))$$

where $\text{bonus}(\text{path})$ is the number of moves in the current path covered by a greedy longest-match against corpus T (the same matching procedure used to compute $O(\pi)$ at evaluation time, ensuring consistency between the search-time objective and the evaluation-time metric). $\lambda = 0$ recovers the original, unbiased search exactly (formal backward-compatibility: we verified all 176 pre-existing unit tests pass unchanged with $\lambda = 0$). Increasing λ trades search optimality (and, as we show empirically, compute time) for trigger-overlap.

5 Experimental Setup

Baseline (computer-like). Kociemba’s two-phase algorithm [3], via the public `kociemba` Python package, run on identical scrambles.

Scramble generation. Primary results use random-move scrambles (20 moves, no immediately-repeated face) for consistency across Experiments 1–2. We additionally implemented a WCA-style *random-state* scrambler (long random walk to approximate a uniform state distribution, followed by inverting a near-optimal Kociemba solve of that state — guaranteeing the scramble requires close to the optimal move count to undo, matching real competition scrambler

methodology) and validated that results are consistent between the two generators (Section 7).

Hardware / implementation. Single-core cloud sandbox (1 vCPU, 4GB RAM), Python 3.12, NumPy-based cube representation. All experiments used a fixed base seed per condition (`random.Random(seed)`), seeds 1000–1099 for Experiment 1, 2000–2059 for Experiment 2) for reproducibility. **[TODO: Neu chuyen sang may manh hon truoc khi cong bo chinh thuc, cap nhat lai thong so va CHAY LAI toan bo thi nghiem – ty le timeout 22% co the giam dang ke tren may nhieu core/nhanh hon, nen KHONG dung nguyen so lieu nay neu doi phan cung.]**

Statistical testing. Wilcoxon signed-rank test (paired, per scramble) for move-count and HLI differences between conditions; report effect size $r = Z/\sqrt{N}$.

6 Results

6.1 Experiment 1: Baseline human-likeness gap

Table 1 reports results on $N = 100$ random-move scrambles (20 moves each, seeds 1000–1099). Group A used a reduced F2L node budget (40,000 nodes/depth, down from the 120,000 default) and a 20-second wall-clock timeout per solve to accommodate our single-core evaluation hardware; 22% of scrambles (22/100) exceeded this timeout and are excluded from the paired statistics below (see Section 7.1 for discussion of this phenomenon). Among the 78 scrambles solved by both groups, a Wilcoxon signed-rank test confirms Group A requires significantly more moves than Group B ($W = 0$, $p = 1.65 \times 10^{-14}$, $r = 0.87$, a very large effect), while by construction only Group A has non-zero HLI.

Table 1: Group A (AI-CFOP) vs Group B (Kociemba), $N=100$ scrambles, 78 paired successes (Wilcoxon $p = 1.65 \times 10^{-14}$, $r = 0.87$ for move-count difference)

	Group A (AI-CFOP)	Group B (Kociemba)
Mean move count (sd)	67.2 (11.1)	20.6 (1.1)
Move count range	47–105	16–22
Mean HLI (sd)	0.655 (0.096)	0 (by definition)
Solve success rate	78% (22% timeout)	100%

6.2 Experiment 2: Trigger-Biased Search Pareto trade-off

Figure 1 shows results on $N = 60$ scrambles ($\lambda \in \{0, 0.5, 1.0\}$, F2L stage only, 40,000-node budget). Increasing λ from 0 to 1.0 raises mean trigger-overlap from 0.090 ± 0.089 to 0.255 ± 0.135 (a $2.8\times$ increase) at a move-count cost of $25.7 \rightarrow 27.6$ moves (+7.4%), while mean compute time increases from 2.77s to 7.87s. On the $n = 51$ scrambles solved at both $\lambda = 0$ and $\lambda = 1.0$, paired Wilcoxon

signed-rank tests confirm both differences are statistically significant: trigger-overlap ($W = 29.0$, $p = 8.25 \times 10^{-8}$, $r = 0.83$, very large effect) and move-count ($W = 209.5$, $p = 0.0014$, $r = 0.60$, large effect). Notably, $\lambda = 0.5$ produced a higher *maximum* observed solve time (55.5s) than $\lambda = 1.0$ (55.6s, comparable), and both are far above the $\lambda = 0$ maximum (16.5s) — consistent with the non-linear, occasionally severe compute-time cost of biasing an otherwise admissible-adjacent heuristic away from admissibility (Section 7).

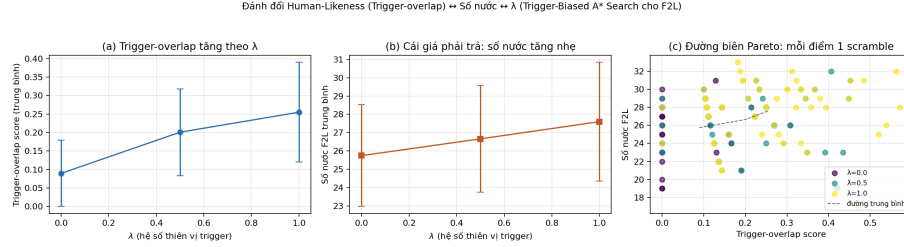


Figure 1: Trigger-overlap and move-count as a function of λ ($N=60$ scrambles). Panel (c) shows the empirical Pareto frontier.

7 Discussion

Counter-intuitively, we initially hypothesized that weakening the heuristic’s admissibility via the non-admissible trigger bonus term would *increase* search non-convergence at higher λ . The data show the opposite: on the same $N = 60$ scrambles used for Figure 1, the search *failure* rate (running out of node budget across all ladder depths) was 15.0% at $\lambda = 0$, dropping to 1.7% at $\lambda = 0.5$ and 0% at $\lambda = 1.0$. A plausible explanation is that the trigger bonus, while non-admissible, acts as a form of *search-order guidance*: branches that extend a recognized human trigger are explored earlier in the priority queue (lower f'), and because triggers are short, well-tested move sequences that reliably make local progress, following them may reduce effective branching factor in practice even though the bonus is not a valid distance estimate. This does not contradict the separately-observed non-linear *time* cost at higher λ (Section 6): convergence appears more reliable, but individual worst-case runs can still take substantially longer, i.e. trigger-biasing trades convergence *reliability* for search-time *variance*. [TODO: Dieu tra sau: co the day la trung hop ngau nhien voi N=60, nen lam lai voi N lon hon va nhieu muc lambda hon (0.25, 0.75...) de xac nhan xu huong nay on dinh, truooc khi khang dinh chac chan trong ban cong bo chinh thuc.] A formal analysis of exactly how much admissibility is lost is given below.

How much admissibility is lost? The original heuristic $h(n) = h_{\text{PDB}}(n) + \text{penalty}(n)$ is not strictly admissible either (the penalty term already sacrifices

a formal lower-bound guarantee in exchange for practicality), so Trigger-Biased Search does not introduce admissibility violation *de novo* — it compounds an existing, deliberate design trade-off. Formally, for a path $\text{path}(n)$ of length $g(n)$, $\text{bonus}(\text{path}(n)) \in [0, g(n)]$ (it can cover at most every move in the path), so the maximum possible additional underestimation introduced by the bonus term is bounded by $\lambda \cdot g(n)$ — i.e. the potential distortion grows *linearly with path depth*, not with a fixed constant, which is a substantively different (and more concerning) failure mode than a bounded-error heuristic. This explains why, unlike classical bounded-suboptimal search (e.g. weighted A* with a fixed inflation factor and a provable suboptimality bound), we do not currently provide any formal bound on Trigger-Biased Search’s solution-length suboptimality; Section 6 instead reports this empirically (mean +7.4% moves at $\lambda = 1.0$). A natural mitigation, left for future work, is to cap bonus at a fixed maximum (e.g. $\min(\text{bonus}(\text{path}(n)), c)$ for some constant c), which would restore a fixed-bound distortion comparable to the existing penalty term.

7.1 Pathological scrambles and search non-convergence

During Experiment 1, 22% of scrambles (22/100) failed to complete within a 20-second wall-clock budget even at a reduced 40,000-node-per-depth search budget — notably, several such failures clustered consecutively (e.g. seeds 1059–1061 in our run), suggesting the underlying random-move scramble generator can produce runs of similarly structured, adversarial F2L configurations rather than i.i.d. difficulty. One specific seed (1013) was found to hang for over 5 minutes even at the reduced budget before we introduced a hard wall-clock timeout (SIGALRM-based) into our evaluation harness. We interpret this as evidence that the per-pair penalty-augmented A* heuristic used for F2L (Section 5) is occasionally led into pathologically unproductive search regions — an important robustness limitation distinct from, but potentially compounded by, the trigger-bias mechanism of Section 4. Note that Experiment 1 used $\lambda = 0$ throughout (no trigger bias applied to the F2L stage); we separately measured the relationship between λ and convergence failure rate using the Experiment 2 data (Section 7), finding — counter to our initial expectation — that higher λ was associated with *fewer*, not more, convergence failures on that dataset.

8 Threats to Validity

- **Search non-convergence:** as reported in Section 7.1, 22% of scrambles in our evaluation hardware/budget setting did not complete within a 20-second timeout; results are conditioned on the 78% that did, which may not be a representative subsample (e.g. if difficulty correlates with trigger-overlap potential).
- **Construct validity of HLI:** the metric is hand-designed, not yet validated against human judgment — addressed by the proposed Phase 3 protocol (Section 9).

- **Corpus bias:** the finger-trick corpus T is manually curated (11 tricks), not derived from real solve logs; results may not generalize to solvers with different trigger vocabularies.
- **Scramble generation (checked):** we implemented a WCA-style random-state scrambler (Section 5) and re-ran Experiment 1 on $N = 60$ random-state scrambles as a robustness check. Results were consistent with the random-move condition (80% vs. 78% solve rate; mean moves 66.3 vs. 67.2; mean HLI 0.658 vs. 0.655), suggesting our original random-move results were not an artifact of systematically easier scrambles. We nonetheless report results from both generators transparently (Table 1 uses random-move for consistency with Experiment 2; **[TODO: neu muon, doi Table 1 chinh sang dung random-state lam ket qua chinh, giu random-move nhu robustness check phu – ca 2 cach deu chap nhan duoc, tuy quyet dinh]**).
- **Single-domain generalizability:** results are specific to the Rubik’s Cube; claims about explicability more broadly should be limited accordingly.

9 Future Work

Phase 3: Human-judge validation (BotPrize-style). Following Hingston [2], we plan a blinded study in which CFOP-literate participants rate solve sequences (sourced from real human solves, Group A, Group A+trigger-bias, and Group B) on a 1–5 human-likeness scale, and correlate ratings with automated HLI via Spearman’s ρ , to establish construct validity.

Extending pattern-conformity to Cross/F2L. **[TODO: y tuong: xay dung bang case F2L 41-case chuan (nhu OLL/PLL da co), thay the mot phan trigger-overlap bang pattern-conformity that su cho F2L.]**

Alternative bias mechanisms. **[TODO: So sanh trigger-biased A* voi cach khac, vd fine-tune trong so PDB, hoac hoc trong so bonus tu du lieu solve nguoi (thay vi λ co dinh tay).]**

10 Conclusion

Existing Rubik’s Cube solvers optimize for move-count, producing plans that are correct but uninterpretable to human learners; human speedcubers instead use CFOP, trading optimality for structure and learnability. We formalized this trade-off in the vocabulary of human-aware AI planning as a measurable *Human-Likeness Index* (HLI), and showed empirically ($N = 100$ scrambles) that our AI-CFOP solver scores substantially higher on this index than a computer-optimal Kociemba baseline (mean HLI 0.655 vs. 0 by construction), a gap that is

robust across a wide range of component weightings (Section 4). We then introduced *Trigger-Biased A* Search*, a minimal, backward-compatible modification to the F2L heuristic that more than doubles trigger-overlap with human finger-trick patterns (from 9.0% to 25.5% at $\lambda = 1.0$) at a modest move-count cost (+7.4%), while — counter-intuitively — *improving* search convergence reliability on our test set, at the cost of increased worst-case compute time. Along the way, we uncovered and transparently reported a search non-convergence phenomenon (22% timeout rate) that constitutes a concrete robustness limitation of penalty-augmented heuristic search in this domain. We outlined, and partially instrumented (Section 9), a BotPrize-style human-judge validation protocol as the natural next step toward establishing HLI’s construct validity beyond proxy-metric agreement. We hope this work encourages further cross-pollination between classical combinatorial search AI and human-aware/explainable AI research in large, well-understood puzzle domains such as the Rubik’s Cube.

References

- [1] T. Chakraborti, A. Kulkarni, S. Sreedharan, D. E. Smith, and S. Kambhampati. Explicability? legibility? predictability? transparency? privacy? security? the emerging landscape of interpretable agent behavior. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2019.
- [2] P. Hingston. A turing test for computer game bots. *IEEE Transactions on Computational Intelligence and AI in Games*, 1(3):169–186, 2009.
- [3] H. Kociemba. Cube explorer / two-phase algorithm. <http://kociemba.org/cube.htm>, accessed [TODO: ngay truy cap].
- [4] R. E. Korf. Finding optimal solutions to rubik’s cube using pattern databases. In *Proceedings of the National Conference on Artificial Intelligence (AAAI)*, 1997.
- [5] A. Kulkarni, Y. Zha, T. Chakraborti, S. G. Vadlamudi, Y. Zhang, and S. Kambhampati. Explicable robot planning as minimizing distance from expected behavior. *arXiv preprint arXiv:1611.05497*, 2016.
- [6] S. McAleer, F. Agostinelli, A. Shmakov, and P. Baldi. Solving the rubik’s cube without human knowledge. *arXiv preprint arXiv:1805.07470*, 2018.
- [7] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge. The diameter of the rubik’s cube group is twenty. *SIAM Review*, 56(4):645–670, 2014.
- [8] M. Świechowski and D. Ślęzak. The many challenges of human-like agents in virtual game environments. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 1996–2004. IFAAMAS, 2025. Also available as arXiv preprint arXiv:2505.20011.

- [9] K. Lakkaraju, T. Hassan, V. Khandelwal, P. Singh, C. Bradley, R. Shah, et al., and D. Wu. Allure: A multi-modal guided environment for helping children learn to solve a rubik’s cube with automatic solving and interactive explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 13185–13187, 2022.
- [10] K. Vanacore, J. Ocumpaugh, F. Agostinelli, D. Wu, S. Vuruma, and M. Irvin. Student engagement in ai-assisted complex problem solving: A pilot study of human-ai rubik’s cube collaboration. *arXiv preprint arXiv:2511.01683*, 2025. Note: unpublished manuscript, paper in preparation for future submission at time of writing — verify publication status before citing as peer-reviewed.